

Module 1: Conditions and Sequences

PUBHLT 0411 · Python for Public Health Data Analysis

Soumik Purkayastha

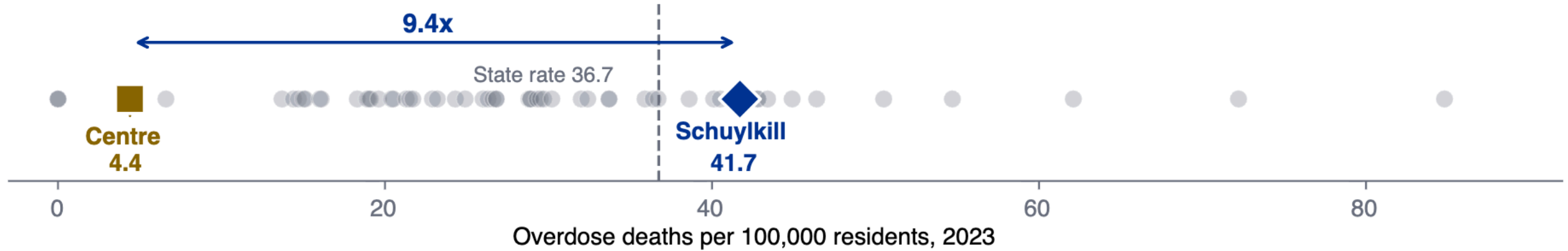
Dept. of Biostatistics and Health Data Science, University of Pittsburgh

October 8, 2026

Two counties.

Two Pennsylvania counties. Populations within 10% of one another. One buries nine times as many residents to overdose as the other.

Schuylkill and Centre counties differ ninefold



Source: PA DOH, Estimated Drug Overdose Deaths, 2023

Figure 1. Centre County, a gold square at 4.4 deaths per 100,000, sits near the low end of the distribution; Schuylkill County, a blue diamond at 41.7, sits in the upper third. An arrow spans the two, labelled 9.4x. The state rate of 36.7 is marked by a dashed line.

The question this session opens

Module 0 established the two rates. **This session takes that rate and implements a classification:** given a county's deaths and population, the code decides whether its burden is low, moderate, or high — and states the evidence.

Required from Module 0: assignment, `int` and `float`, arithmetic, f-strings.

Comparison operators return a boolean

A comparison is an expression. It evaluates to `True` or `False`.

Python Code

↺ Start Over

▶ Run Code

```
1 schuylkill_rate = 41.7
2 centre_rate = 4.4
3
4 print(schuylkill_rate > centre_rate)
5 print(schuylkill_rate == centre_rate)
6 print(schuylkill_rate != centre_rate)
7 print(centre_rate >= 4.4)
```

Operator	Meaning
<code>==</code>	equal to
<code>!=</code>	not equal to
<code><</code> <code>></code>	less than, greater than
<code><=</code> <code>>=</code>	less than or equal, greater than or equal

⚠ `=` assigns; `==` compares

`rate = 41.7` binds a name. `rate == 41.7` asks a question. Substituting one for the other is among the most frequent errors in introductory Python.

Boolean operators combine conditions

Python Code

↺ Start Over

▶ Run Code

```
1 population = 143786
2 overdose_deaths = 60
3 rate_per_100k = 41.7
4
5 print(population > 100000 and rate_per_100k > 36.7)
6 print(population > 200000 or rate_per_100k > 36.7)
7 print(not rate_per_100k > 36.7)
```

1. **and** — **True** only when **both** sides are **True**
2. **or** — **True** when **either** side is **True**
3. **not** — inverts the value

💡 R analogue

R's **&&** and **||** correspond to **and** and **or**. Python spells them as words, and provides no **&&** operator for this purpose.

if executes a block when a condition holds

Python Code

↺ Start Over

▶ Run Code

```
1 rate_per_100k = 41.7
2
3 if rate_per_100k > 36.7:
4     print("Above the Pennsylvania rate")
```

Two pieces of syntax are mandatory:

1. **The colon** ends the `if` line.
2. **The indentation** — four spaces — marks the conditional block.

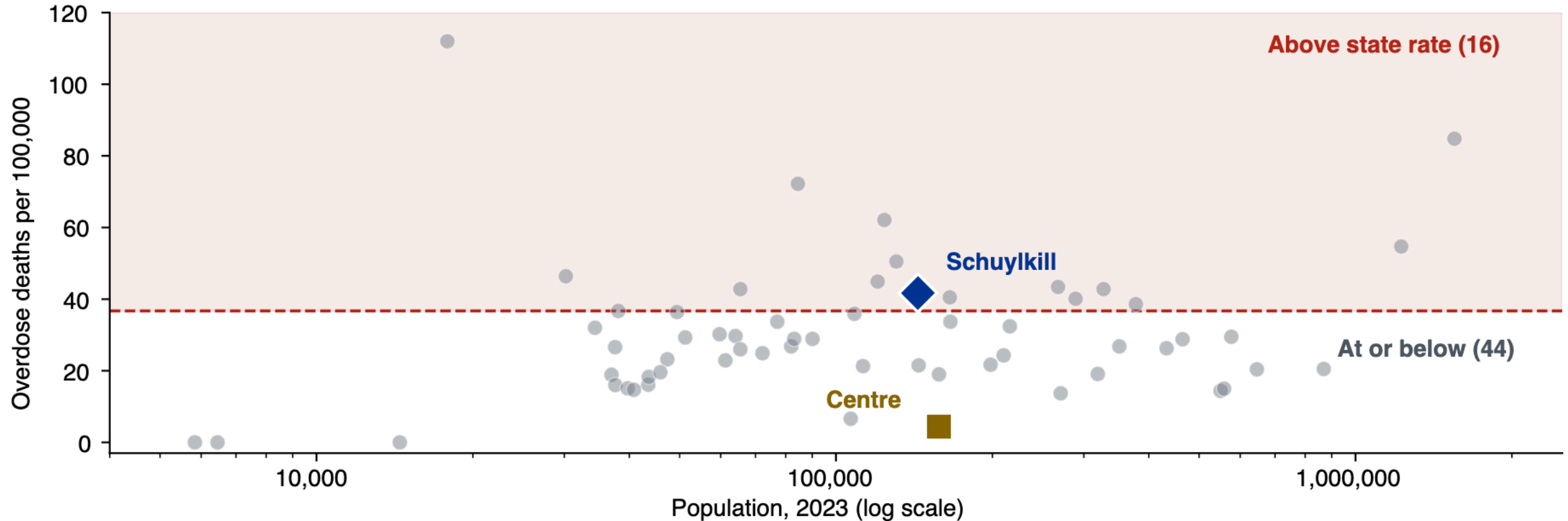
Python Code

↺ Start Over

▶ Run Code

```
1 rate_per_100k = 4.4
2
3 if rate_per_100k > 36.7:
4     print("Above the Pennsylvania rate")
5
6 print("This line runs regardless")
```

One threshold splits the counties in two



Source: PA DOH, Estimated Drug Overdose Deaths, 2023. Seven counties with suppressed counts are not shown.

Figure 2. Each point is a county, with population on a log scale across and overdose rate up. A dashed line at the state rate of 36.7 divides them: 16 counties lie above it, including Schuylkill; 44 lie at or below it, including Centre.

`elif` and `else` handle the remaining cases

Python Code

↺ Start Over

▶ Run Code

```
1 rate_per_100k = 19.0
2
3 if rate_per_100k >= 36.7:
4     print("High")
5 elif rate_per_100k >= 13.1:
6     print("Moderate")
7 else:
8     print("Low")
```

Evaluation stops at the first **True** branch. Order therefore determines behavior.

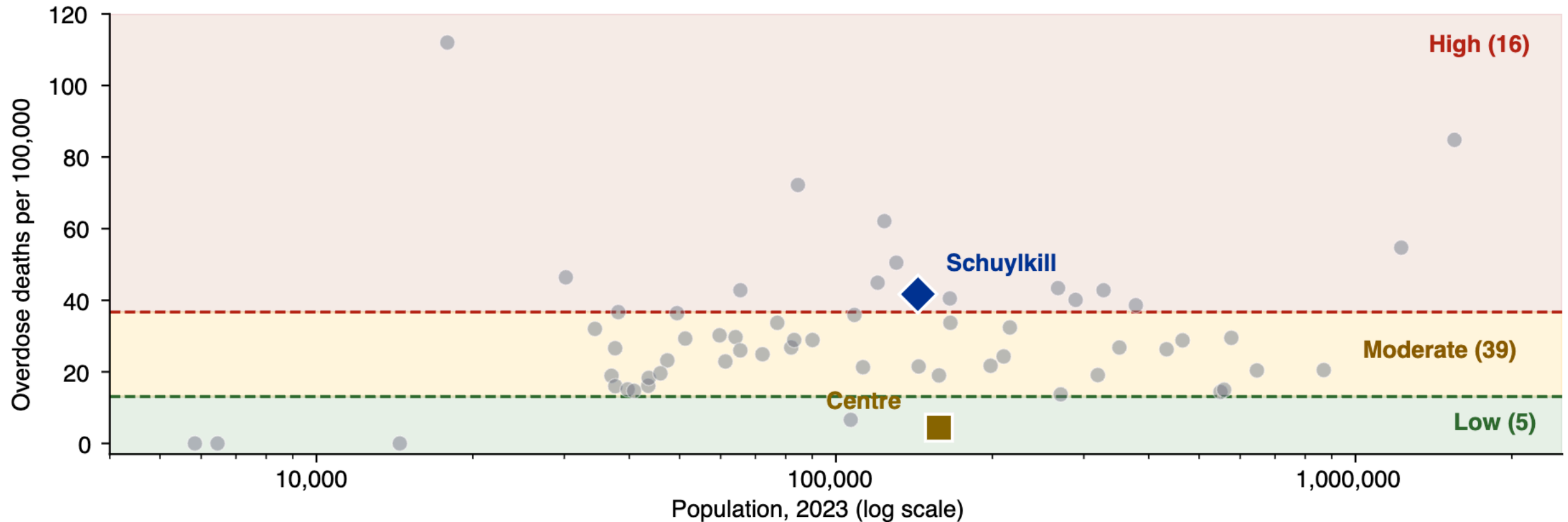
💡 The two thresholds are real:

1. **36.7** per 100,000: Pennsylvania's overdose mortality rate, 2023
2. **13.1** per 100,000: the Healthy People 2030 target

📌 Only one branch executes

Once a condition matches, the remaining branches are not evaluated. A chain ordered from lowest threshold upward reports the wrong category for every value.

Two thresholds give three bands



Source: PA DOH, Estimated Drug Overdose Deaths, 2023. Seven counties with suppressed counts are not shown.

Figure 3. The same counties, now in three bands. Above the state rate of 36.7: High, 16 counties, including Schuylkill. Between the Healthy People 2030 target of 13.1 and the state rate: Moderate, 39 counties. Below 13.1: Low, 5 counties, including Centre.

Checkpoint: the condition tests the wrong quantity

Objective: report whether Schuylkill County exceeds the state rate of 36.7 per 100,000.

Python Code

↺ Start Over

▶ Run Code

```
1 overdose_deaths = 60
2 population = 143786
3 rate_per_100k = overdose_deaths / population * 100000
4
5 if overdose_deaths > 36.7:
6     print("Above the state rate")
7 else:
8     print("At or below the state rate")
```

❗ A silent error: the comparison is valid but meaningless

The code runs and prints an answer. It compares a **count** (60 deaths) against a **rate** (36.7 per 100,000) — quantities in different units. The comparison is arithmetically legal, so no exception is raised.

Resolution: compare `rate_per_100k > 36.7`. The correct answer is the same here, which is what makes the error survive review.

Augmented assignment updates a value in place

Python Code

↺ Start Over

▶ Run Code

```
1 total_deaths = 0
2
3 total_deaths += 60      # Schuylkill
4 total_deaths += 7       # Centre
5 total_deaths += 30      # Franklin
6
7 print(total_deaths)
```

`total_deaths += 60` is equivalent to `total_deaths = total_deaths + 60`.

The same form exists for the other operators: `--`, `*=`, `/=`.

💡 Read the statement as an instruction, not an equation

`total_deaths = total_deaths + 60` is false as algebra and correct as a sequence of steps: evaluate the right side, then rebind the name.

Sequences.

A sequence is a generic term for an ordered collection of items where each element can be efficiently accessed using an integer index

A string is a sequence of characters

Each character occupies a position, and **positions are numbered from zero**.

Python Code

↺ Start Over

▶ Run Code

```
1 county = "Schuylkill"
2
3 print(county[0])
4 print(county[1])
5 print(county[-1])
6 print(len(county))
```

1	S	c	h	u	y	l	k	i	l	l
2	0	1	2	3	4	5	6	7	8	9
3	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

💡 The first position is `0`, and the last is `len() - 1`

`county[10]` raises `IndexError: string index out of range`, because the valid positions of a ten-character string are `0` through `9`. Negative indices count backward from the end.

A slice extracts a range of positions

`sequence[start:stop]` includes `start` and **excludes** `stop`.

Python Code

↺ Start Over

▶ Run Code

```
1 county = "Schuylkill"
2
3 print(county[0:4])
4 print(county[:4])
5 print(county[4:])
6 print(county[::2])
```

1. Omitting `start` begins at the beginning
2. Omitting `stop` continues to the end
3. A third value is the **step**: `[::2]` takes every second character

💡 The stop position is excluded

`county[0:4]` yields `Schu` — four characters, positions 0 through 3. The length of a slice is `stop - start`, which is the reason the convention is convenient.

String methods transform text

A method is called on the value itself, using a dot.

Python Code

↺ Start Over

▶ Run Code

```
1 county = "  Schuylkill \n"
2
3 print(repr(county.strip()))
4 print(county.strip().upper())
5 print("Mc Kean".replace(" ", ""))
```

Method	Effect
<code>.strip()</code>	removes leading and trailing whitespace
<code>.upper()</code>	converts to uppercase
<code>.replace(a, b)</code>	substitutes every occurrence of <code>a</code> with <code>b</code>
<code>.split(sep)</code>	divides into a list at each separator

Methods return new values

`county.upper()` produces a new string. The original binding is unchanged unless it is reassigned.

`.split()` divides a line into fields

Python Code

↺ Start Over

▶ Run Code

```
1 line = "Schuylkill,2023,60,143786,41.7"
2
3 fields = line.split(",")
4 print(fields)
5 print(fields[0])
6 print(int(fields[2]))
```

The result is a **list** — an ordered sequence of values, indexed from zero exactly as a string is.

⚠ Fields extracted from text are text

`fields[2]` is the string `"60"`, not the number `60`. Arithmetic requires explicit conversion with `int()` or `float()`.

A list holds an ordered sequence of values

Python Code

↺ Start Over

▶ Run Code

```
1 counties = ["Schuylkill", "Centre", "Franklin"]
2 rates = [41.7, 4.4, 19.0]
3
4 print(counties[0], rates[0])
5 print(counties[-1], rates[-1])
6 print(len(counties))
7 print(counties[0:2])
```

Lists are indexed and sliced with the same notation as strings. **Position `i` in `counties` corresponds to position `i` in `rates`** — the two lists are parallel.

💡 R analogue

An R vector, with one difference: a Python list may hold values of mixed type, and it does not recycle.

Lists are modified in place

Python Code

↺ Start Over

▶ Run Code

```
1 counties = ["Schuylkill", "Centre"]
2 counties.append("Franklin")
3
4 print(counties)
5 print(len(counties))
6 print("Centre" in counties)
7 print("Erie" in counties)
```

1. `.append(value)` adds one value to the end
2. `value in sequence` tests membership and returns a boolean

 `.append()` returns nothing

The list is modified where it stands. Writing `counties = counties.append("Franklin")` rebinds the name to `None` and discards the list.

Checkpoint: the block is not indented

Objective: print a warning for a county above the state rate.

Python Code

↺ Start Over

▶ Run Code

```
1 rate_per_100k = 41.7
2
3 if rate_per_100k > 36.7:
4 print("Above the state rate")
```

❗ **IndentationError** — the block is missing

The reported message is `expected an indented block after 'if' statement on line 3`.

A colon at the end of a line commits the following line to be indented. Python defines the conditional block by that indentation, so an unindented line leaves the `if` with no body.

Resolution: indent the `print()` by four spaces.

Is a county in need of assistance?

Population size and overdose rate together determine how resources are allocated — and a county's classification is the first step in that decision.

Exercise: classify a county by its rate

Franklin County, 2023: 30 deaths, population 157,854.

Exercise [↺ Start Over](#) [💡 Show Hint](#)

[▶ Run Code](#)

```
1 county = "Franklin"
2 overdose_deaths = 30
3 population = 157854
4
5 rate_per_100k = overdose_deaths / population * 100000
6
7 if rate_per_100k >= 36.7:
8     classification = "High"
9 elif rate_per_100k >= ____:
10     classification = "Moderate"
11 else:
12     classification = "Low"
13
14 print(f"{county}: {rate_per_100k:.1f} per 100,000 – {classification}")
```

The classification applied to both counties

Python Code

↺ Start Over

▶ Run Code

```
1 counties = ["Schuylkill", "Centre"]
2 deaths = [60, 7]
3 populations = [143786, 157795]
4
5 schuylkill_rate = deaths[0] / populations[0] * 100000
6 centre_rate = deaths[1] / populations[1] * 100000
7
8 print(f"{counties[0]}: {schuylkill_rate:.1f} per 100,000")
9 print(f"{counties[1]}: {centre_rate:.1f} per 100,000")
10 print(f"Ratio: {schuylkill_rate / centre_rate:.1f}x")
11
12 ✓ if schuylkill_rate >= 36.7:
13     print(f"{counties[0]} is classified High")
14 ✓ if centre_rate < 13.1:
15     print(f"{counties[1]} is classified Low")
```

A rate needs enough deaths to be trusted

Python Code

↺ Start Over

▶ Run Code

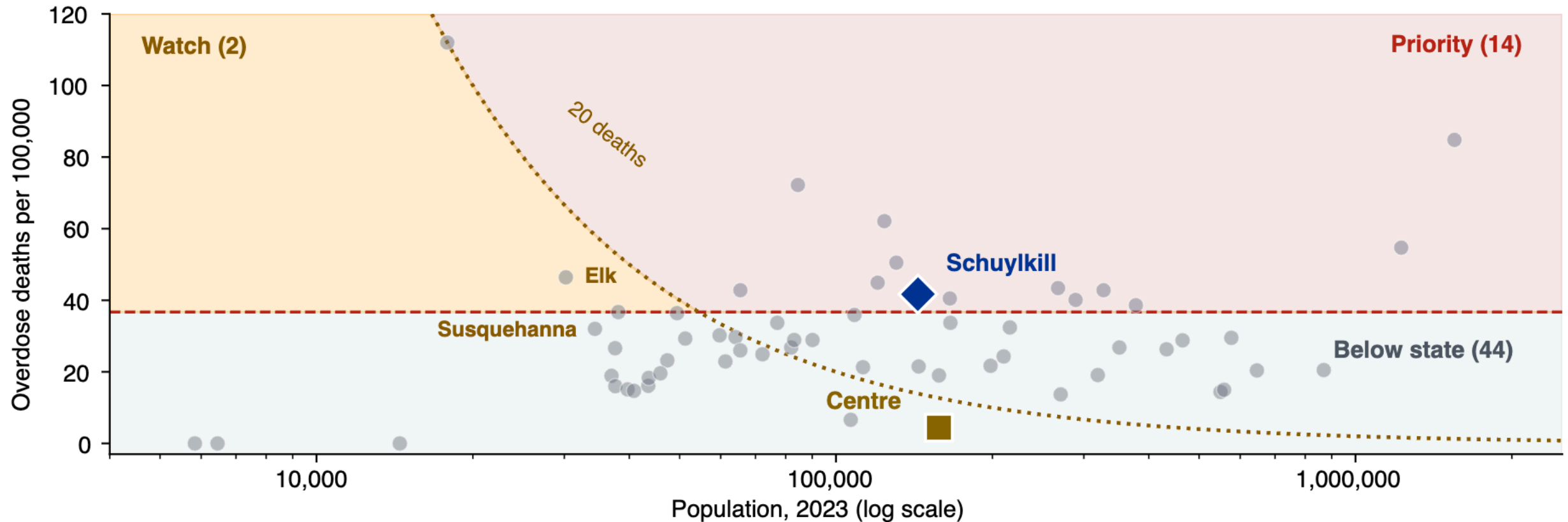
```
1 county = "Elk"
2 overdose_deaths = 14
3 population = 30198
4 rate_per_100k = overdose_deaths / population * 100000
5
6 if rate_per_100k >= 36.7 and overdose_deaths >= 20:
7     classification = "Priority"
8 elif rate_per_100k >= 36.7:
9     classification = "Watch"
10 else:
11     classification = "Below state"
12
13 print(f"{county}: {rate_per_100k:.1f} per 100,000, {overdose_deaths} deaths")
14 print(classification)
```

The second variable is the count behind the rate. Elk's rate is high, but it rests on 14 deaths: one more or one fewer moves it by more than 3 per 100,000.

💡 This threshold is real too

The National Center for Health Statistics marks a death rate as **unreliable when it is based on fewer than 20 deaths**.

Two variables give three classes



Source: PA DOH, Estimated Drug Overdose Deaths, 2023. Seven counties with suppressed counts are not shown.

Figure 4. The same counties, classified on rate and count together. A dotted curve marks where a county has exactly 20 deaths; counties to its left have fewer. Above the state rate and right of the curve: Priority, 14 counties, including Schuylkill. Above the state rate but left of the curve: Watch, 2 counties, Elk and Susquehanna. Below the state rate: 44 counties, including Centre, whose rate rests on 7 deaths.

Session outcomes

1. Evaluating comparisons and combining them with `and`, `or`, `not`
2. Branching with `if` / `elif` / `else`, and the **colon and four-space indent**
3. Accumulating with `+=`
4. Indexing and slicing strings and lists from **position zero**
5. Applying `.strip()`, `.upper()`, `.replace()`, and `.split()`
6. Building lists, extending them with `.append()`, and testing membership with `in`

Preparation for the lab, Tuesday 10/13

The lab classifies additional counties, parses real record lines, and builds parallel lists. No new syntax is required beyond this session.

Module 2 addresses scale: sixty-seven counties and forty-nine months, without repetition.

Final exercise: parse and classify

One line of the 2023 file, as text. Split it, convert the fields, and classify the county.

Exercise [↺ Start Over](#) [💡 Show Hint](#)

[▶ Run Code](#)

```
1 line = "Montour,2023,20,17860"
2 fields = line.split(_____)
3 county = fields[0]
4 overdose_deaths = int(fields[2])
5 population = int(fields[____])
6 rate_per_100k = overdose_deaths / population * 100000
7 if rate_per_100k >= 36.7 and overdose_deaths >= 20:
8     classification = "Priority"
9 elif rate_per_100k >= 36.7:
10     classification = "Watch"
11 else:
12     classification = "Below state"
13 print(f"{county}: {rate_per_100k:.1f} on {overdose_deaths} deaths – {classification}")
```

💡 Your turn!

Centre's rate of 4.4 rests on 7 deaths. Does "ninefold" still stand?

💡 Your turn!

Montour has exactly 20 deaths. Should `>= 20` or `> 20` decide its class?

That's all folks!

See you on Tuesday!